

How to gain access to your AWS Account

Hackathon

Register Account

Construct your username

- **Registration of Account is performed by Group representatives only** — just one person per team signs in and manages the account
- Your username is the email you registered as group leader, with hackathon2026, added in front.
- Note the **comma**, and **no spaces**.
- Example: registered email bella.tan@example.com → username: hackathon2026,bella.tan@example.com
- Enter this exact username on the AWS access portal sign-in page (see next slide).

Verification Code

Go to this URL in your browser: <https://d-9667b91afb.awsapps.com/start>



Sign in

Username

Next

By continuing, you agree to the [AWS Customer Agreement](#) or other agreement for AWS services, and the [Privacy Notice](#). This site uses essential cookies. See our [Cookie Notice](#) for more information.

Enter your username (see previous slide on how to construct it from your email address) and click 'Next'.



Additional verification required

Username:
hackathon2026[redacted]
[redacted] (not you?)

If the username you entered was correct, you will receive an email with a verification code. Enter it below to verify your identity.

Verification code

Sign in

[Resend code](#)

aws

Verify your identity

Please enter this code to verify your identity and complete your sign-in to AWS.

Verification code

[redacted]

(This code is valid for 10 minutes)

For additional help, [contact AWS Support](#).

Amazon Web Services will never email you and ask you to disclose or verify your password, credit card, or banking account number. If you receive a suspicious email with a link to update your account information, do not click on the link. Instead, report the e-mail to Amazon Web Services for investigation.

Check your email inbox. You should receive a verification code. Enter it and press 'Sign in'.

Set up 2FA



Register MFA device

Username:

hackathon2026,tingyewk+sandboxtest0001@gmail.com (not you?)

Your organization requires multi-factor authentication (MFA) for added security during sign-in. Each time you sign in, you'll be prompted for your password and an MFA device.

[Learn more](#)

Select one of the options below to get started:

- ☒  **Authenticator app**
Authenticate using a code generated by an app installed on your mobile device or computer.
- ☐  **Security key**
Authenticate by touching a hardware security key such as YubiKey, Feitian, etc.
- ☐  **Built-in authenticator**
Authenticate using a fingerprint scanner or camera built-in to your computer such as Apple TouchID, Windows Hello, etc.

Next

You will be greeted with a 'Register MFA device' page. Select 'Authenticator app' and click 'Next'.

Set up Authenticator App



Set up the authenticator app

Username:
hackathon2026,tingyewk+sandboxtest0001@gmail.com (not you?)

[Back to MFA device options](#)

1



Install either the Google Authenticator, Duo Mobile, or Authy app on your mobile device or computer. [See a list of compatible apps](#)

2

Show QR code

Use your virtual MFA app or your device's camera to scan the QR code. [Show secret key](#)

3

Enter the six digit code from your authenticator app

Authenticator code

Assign MFA

2

Show QR code

Use your virtual MFA app or your device's camera to scan the QR code. [Hide secret key](#)

CTIN5X7V

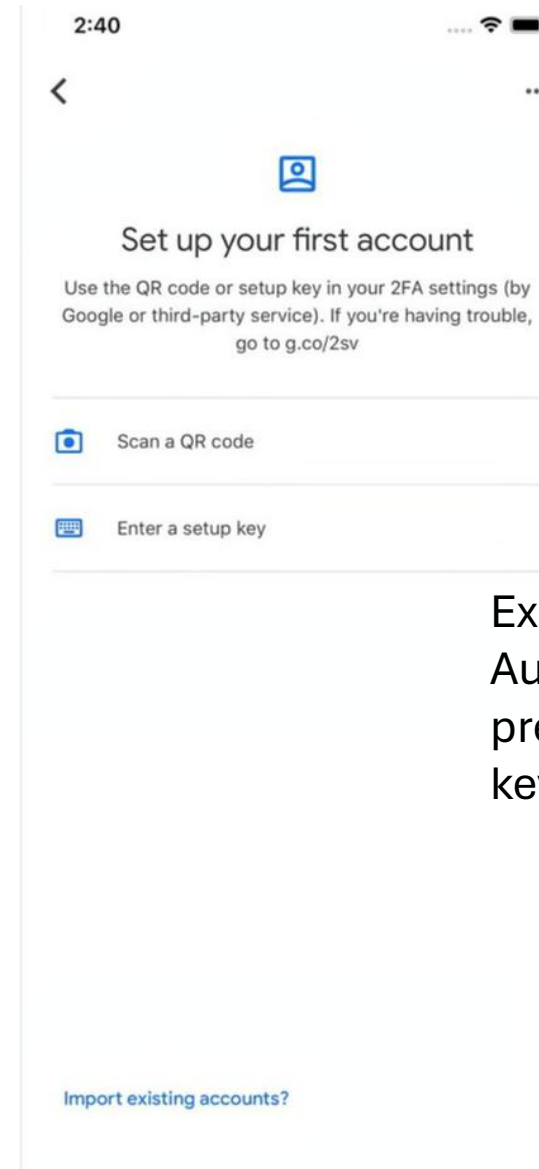


Copied

Click on 'Show secret key'. We recommend that you share this key with your group members so that they can individually log in to the same account without having to bother you. Each teammate adds it to their own authenticator app.

Add 2FA Secret String to your Authenticator App

- Many popular authenticator apps support adding string, you can use your personal apps
- If you don't wish to use an app, you can download a browser (e.g. on Firefox, or Chrome) extension for 2FA



Example: In Google Authenticator, you can press 'Enter a setup key'.

Complete 2FA Setup



Set up the authenticator app

Username:
hackathon2026,tingyewk+sandboxtest0001@gmail.com (not you?)

[Back to MFA device options](#)

1



Install either the Google Authenticator, Duo Mobile, or Authy app on your mobile device or computer. [See a list of compatible apps](#)

2

Show QR code

Use your virtual MFA app or your device's camera to scan the QR code. [Hide secret key](#)

CTIN5X7VEYINC4X2XIIJDDK3Q624RSQZ

3

Enter the six digit code from your authenticator app

Authenticator code

Assign MFA

For Group representative:
Enter your six-digit code, and press 'Assign MFA'.



Authenticator app registered

✓ Your authenticator app has been successfully registered. You can now use it when prompted for additional verification at sign in.

[Rename](#)

Type and description: Authenticator app

Done

You should see this screen when all is successful. Click on 'Done'.

Choose your password



Choose your password

✔ Email address has been successfully verified. ✕

Your password provides you with sign in access to AWS access portal, so it's important we get it right.

Password

Confirm password

☐ Show password

Set new password

Use:

- ... Must be between 8-64 characters
- ... Use upper and lower case letters
- ... Use a number
- ... Use a symbol

Enter a password.

Note that you would have to share this with your group members, so that they can log in as well.

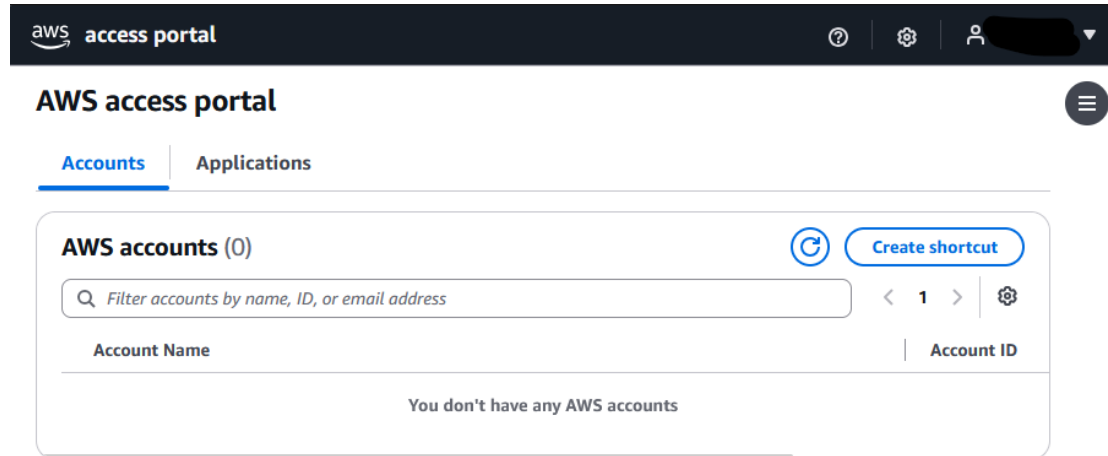
Lease a Sandbox AWS Account

AWS Access Portal

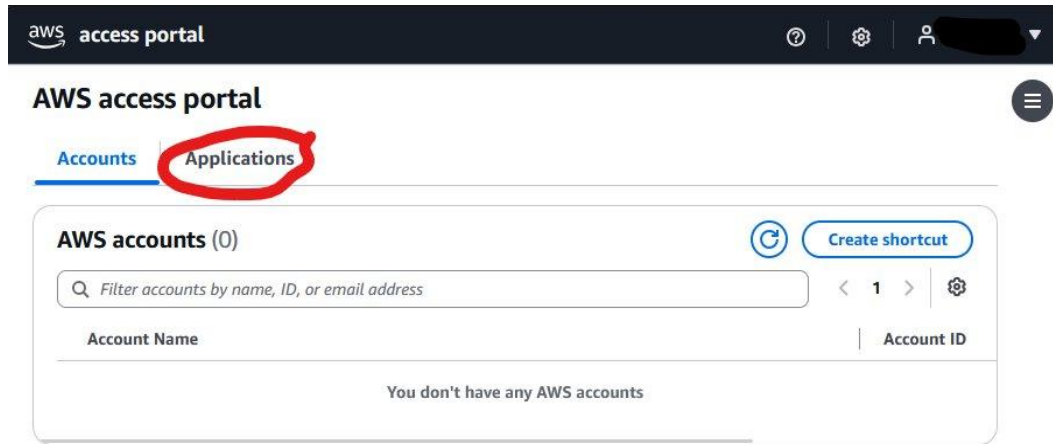
At this point, you have successfully registered a user account for the Group.

Provided the Group Representative has shared the username, password, and 2FA secret, every member in the group should be able to log in and see the 'AWS access portal' on the left.

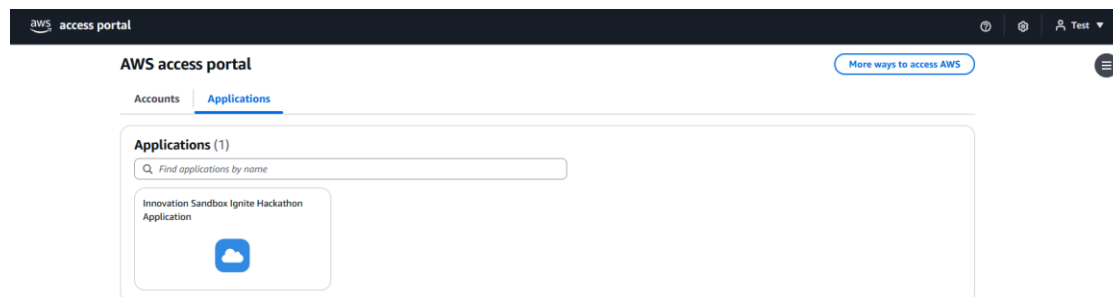
This section will guide you on how to lease a AWS account that will stay with you throughout the hackathon. Just like the user account registration in the previous section, leasing an account is a **one-time action**. Only one member of the group has to do it.



Access Portal

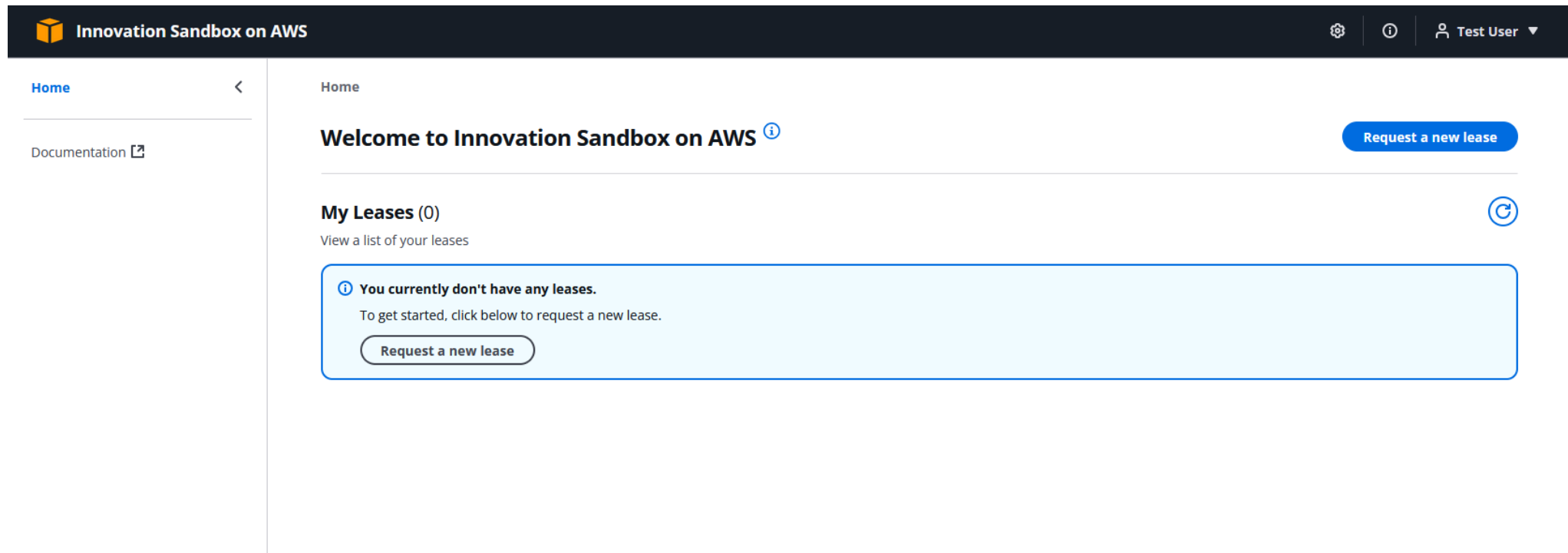


Click on 'Applications' tab.



Under 'Applications' tab, click on **Innovation Sandbox Ignite Hackathon Application**.

Innovation Sandbox on AWS Portal



Click on **Request a new lease.**

Request Lease

[Home](#) > Request lease

Request to lease an AWS sandbox account.

Step 1
Select lease template

Step 2
Terms of Service

Step 3
Review & Submit

Select lease template

What lease template would you like to use?

Hackathon 2026

Description
No description

Blueprint
No Blueprint

Max Budget
\$30

Expires
after 840 hours

Visibility
Public

Approval
Requires approval

[Cancel](#) [Next](#)

Select the lease template 'Hackathon 2026' and click **Next**.

Note: The **lease name**, **Max Budget** and **Expires** fields might show a different value from what you see here.

Review Terms of Service

Request lease

Request to lease an AWS sandbox account.

- Step 1
● Select lease template
- Step 2
● **Terms of Service**
- Step 3
○ Review & Submit

Terms of Service

Before continuing, please review the terms of service below.

By requesting an account, your team agrees to these terms. This is a temporary, shared, cost-capped AWS environment for the hackathon only.

- * **Budget: US\$20 total** for your team – it does not reset, renew, or top up.
- * You'll get an **email warning near US\$12**. At **US\$20** the account freezes (no new or changed resources; existing ones kept briefly for review). If spend reaches **US\$30**, everything may be permanently deleted.
- * **Check your spam/junk folder.** Info and budget-warning emails may land there – add the sender to your contacts or mark it "not spam" so you don't miss a warning.
- * Your lease is **one-time**. It cannot be ended early, extended, reset, or reissued, and approval is required – if your first lease ends because the budget ran out, a repeat request will not be approved. **Managing the cost is your team's responsibility.**
- * **Watch your spend and keep anything you want outside this account.** The environment is wiped when the lease ends or the budget runs out, and there's no way to recover it after. If you're nowhere near the limit you don't need to do much, but if you're approaching it, push your code to your own repo and download any results before the account is nuked.
- * The whole team **shares one account and one budget**; don't share access outside your team.
- * Use it only for your approved project. No real/personal/company data, no crypto mining, no attacking or accessing systems you're not authorized to. Activity is logged.

Requesting a lease confirms your team has read and accepted these terms.

☐ I accept the above terms of service.

Cancel

Previous

Next

Review the Terms of Service.

If you accept, check 'I accept the above terms of service' and click on **Next**.

Review and Submit your Lease

Request lease

Request to lease an AWS sandbox account.

- Step 1
Select lease template
- Step 2
Terms of Service
- Step 3
Review & Submit

Review & Submit

Lease Template Selected

Name
Hackathon 2026

Description

Blueprint
[No Blueprint](#)

Duration
840 hours
after approval

Max Budget
\$30

Approval
[Requires approval](#)

Comments

Optional - add additional comments to support your request

Enter any additional comments...

Cancel

Previous

Submit request

NOTE

Even though Max Budget might show **\$30** on the left, do be warned that the actual budget is set to \$20. At \$20, access to your AWS account will be revoked. At \$30, your AWS account would be terminated to eliminate run-away costs.

Barring certain exceptions, request for additional leases (past the first one) would not be granted. Every group is expected to lease one (and only one maximum) AWS account throughout the hackathon. This is for fairness among all the hackathon groups and to mitigate abuse of resources as well.

Click on **Submit request**.

Wait for approval

Hackathon 2026

Pending Approval

AWS Account ID

 No account assigned yet

Expiry

840 hours
after approval

Budget

\$0 of \$30

0%

 Your account is pending approval

Please wait up to 2 working days for approval. During the start of the hackathon, our team has anticipated the surge of lease requests and thus is prepared to approve (legitimate) requests as swiftly as possible.

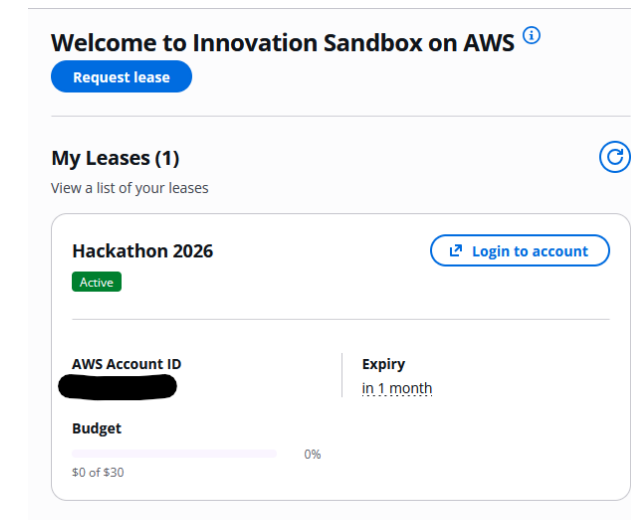
Approval Email

Upon approval from our team, you will receive an email.

It is highly likely that it will land in your spam box. Nevertheless, the email is simply informational.

If you received this email, it means that your request has been approved and you and your group members can proceed.

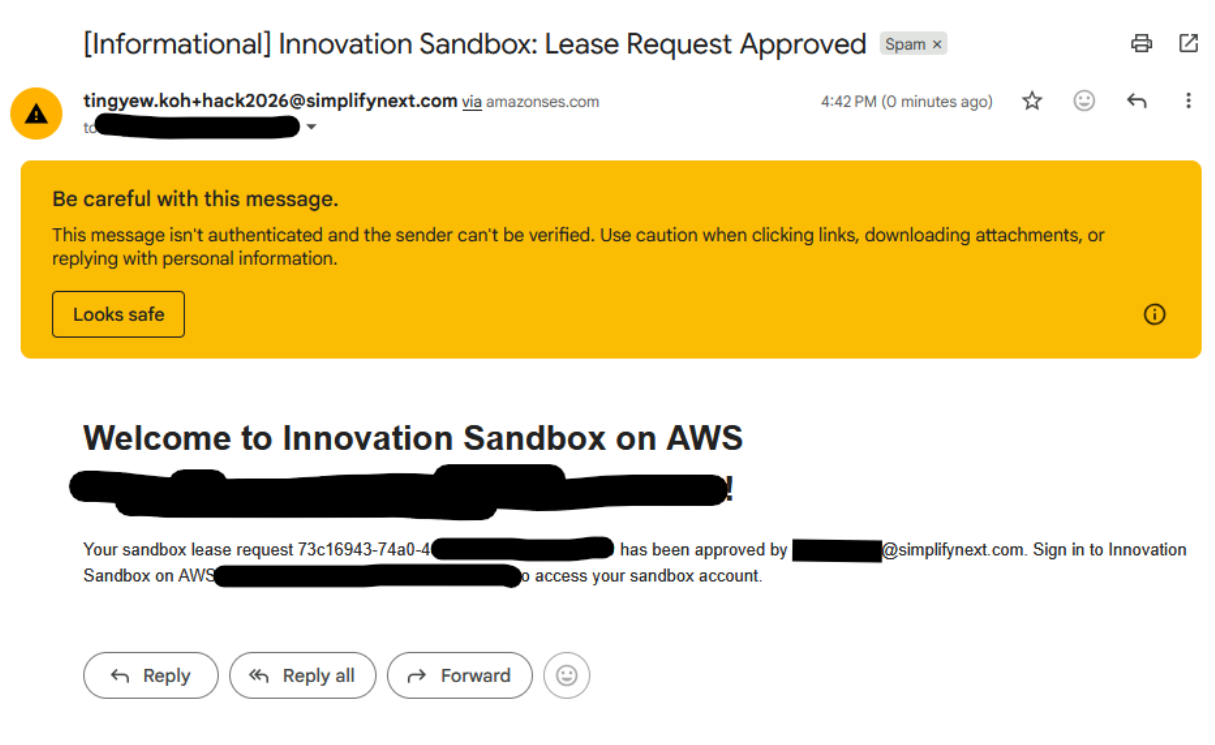
If you did not receive it, please be patient. It is possible that the email has been blocked entirely, so log in to your user account every now and then to check the status.



The screenshot shows the 'Welcome to Innovation Sandbox on AWS' header with a 'Request lease' button. Below is the 'My Leases (1)' section, which includes a 'Login to account' button. The lease details for 'Hackathon 2026' are as follows:

Hackathon 2026	
Active	
AWS Account ID	Expiry
[Redacted]	in 1 month
Budget	
\$0 of \$30	0%

You can keep logging in every now and then to check for the status of your Lease. Please be patient! We will try to approve as quickly as possible.



The email header shows the subject '[Informational] Innovation Sandbox: Lease Request Approved' and the sender 'tingyew.koh+hack2026@simplifynext.com via amazonses.com'. A yellow warning banner states: 'Be careful with this message. This message isn't authenticated and the sender can't be verified. Use caution when clicking links, downloading attachments, or replying with personal information. Looks safe'. The main body of the email reads: 'Welcome to Innovation Sandbox on AWS [Redacted]! Your sandbox lease request 73c16943-74a0-4[Redacted] has been approved by [Redacted]@simplifynext.com. Sign in to Innovation Sandbox on AWS [Redacted] to access your sandbox account.' The footer contains 'Reply', 'Reply all', 'Forward', and an emoji button.

Login to Account

Home

Welcome to Innovation Sandbox on AWS 

[Request a new lease](#)

My Leases (4)

View a list of your leases

Hackathon Basic

Active

[Login to account](#)

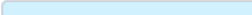
AWS Account ID

004304087990

Expiry

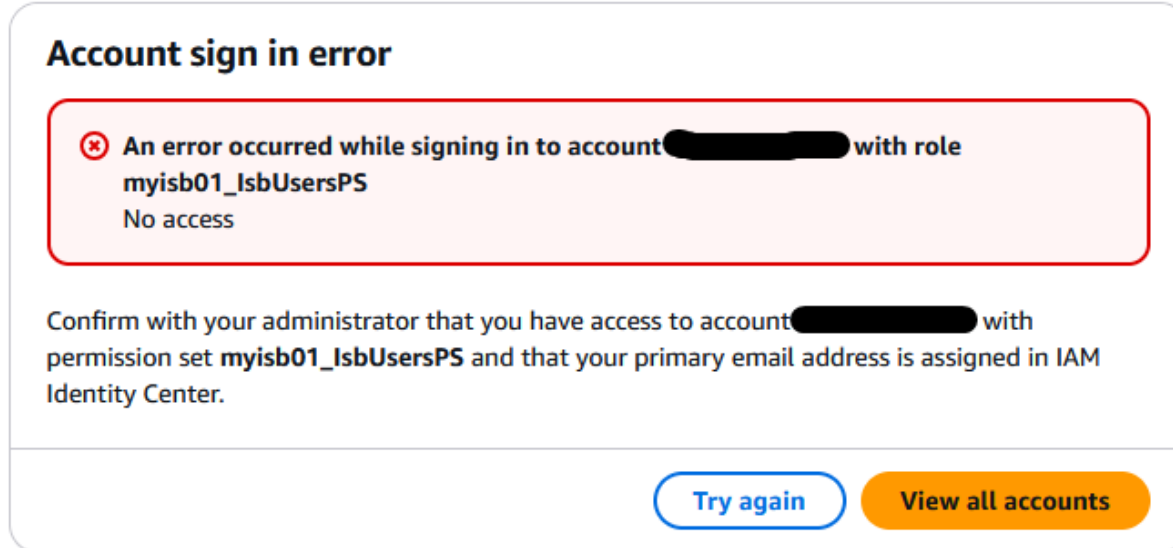
in a month

Budget

 \$100
\$0

You can click on **Login to account**

If error, just wait



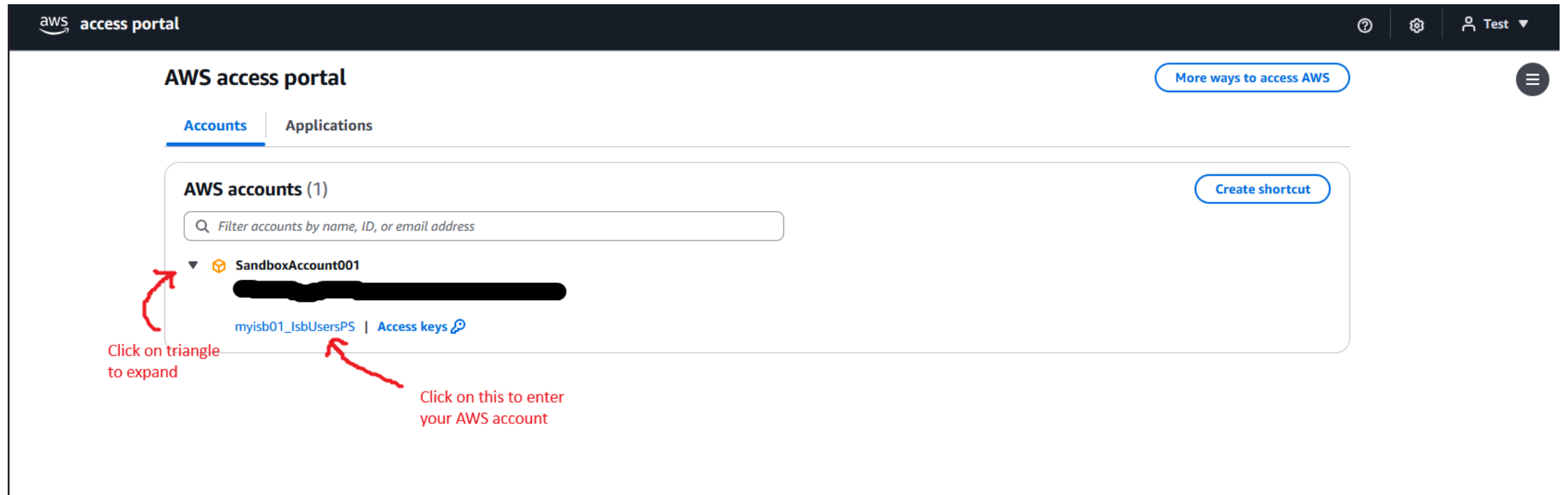
If you see this error, just wait a few minutes and then **Try again**.

Welcome to Innovation Sandbox on AWS (ISB) [redacted]

Your sandbox lease request [redacted] has been auto approved. Please log into the Innovation Sandbox web application [https://\[redacted\].cloudfront.net](https://[redacted].cloudfront.net) to access your sandbox account.

You should receive an email as seen above (check your spam box!) within a few minutes. You may log in after receiving this email.

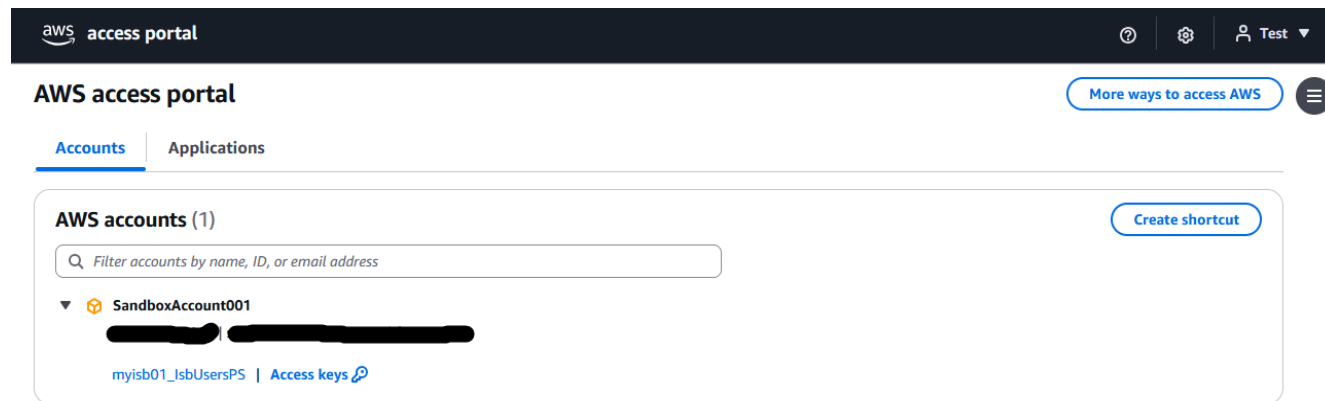
Alternative way of logging in to Account



Once your lease is approved, you can access your AWS account through AWS Access Portal (from earlier). Go to the access portal, under 'Accounts' tab, where previously it was empty, now there should be 1 AWS account.

Access Keys

Retrieve your access keys



Expand your Sandbox Account (triangle), then click on **Access keys** to reveal your credentials.

⚠ Note that these keys are set to expire every 12 hours, so you have to re-login to get new keys.

Get credentials for myisb01_IsbUsersPS

Create access for the account **SandboxAccount001** with **myisb01_IsbUsersPS**. Use any of the following options to access AWS resources programmatically or from the AWS CLI. You can retrieve credentials as often as needed.

[macOS and Linux](#) | [Windows](#) | [PowerShell](#)

▼ AWS IAM Identity Center credentials (Recommended)

To extend the duration of your credentials, we recommend you configure the AWS CLI to retrieve them automatically using the [aws configure sso](#) command. [Learn more](#)

SSO start URL
SSO Region

▼ Option 1: Set AWS environment variables

Run the following commands in your terminal to set the AWS environment variables. [Learn more](#)

```
export AWS_ACCESS_KEY_ID=  
export AWS_SECRET_ACCESS_KEY=  
export AWS_SESSION_TOKEN=
```

▼ Option 2: Add a profile to your AWS credentials file

Copy and paste the following text in your AWS credentials file (`~/.aws/credentials`). [Learn more](#)

```
[  
aws_access_key_id=  
aws_secret_access_key=  
aws_session_token=
```

▼ Option 3: Use individual values in your AWS service client

Copy and paste these values into your code. [Learn more](#)

AWS access key ID
AWS secret access key
AWS session token

#1: Export the keys into your environment

Get credentials for myisb01_IsbUsersPS

Create access for the account **SandboxAccount001 (096863133167)** with **myisb01_IsbUsersPS**. Use any of the following options to access AWS resources programmatically or from the AWS CLI. You can retrieve credentials as often as needed.

[macOS and Linux](#) | [Windows](#) | [PowerShell](#)

▼ AWS IAM Identity Center credentials (Recommended)

To extend the duration of your credentials, we recommend you configure the AWS CLI to retrieve them automatically using the [aws configure sso](#) command. [Learn more](#)

SSO start URL

https://[REDACTED]

SSO Region

ap-southeast-1

▼ Option 1: Set AWS environment variables

Run the following commands in your terminal to set the AWS environment variables. [Learn more](#)

```
export AWS_ACCESS_KEY_ID="[REDACTED]"
export AWS_SECRET_ACCESS_KEY="[REDACTED]"
export AWS_SESSION_TOKEN="[REDACTED]"
```

▼ Option 2: Add a profile to your AWS credentials file

Copy and paste the following text in your AWS credentials file (`~/.aws/credentials`). [Learn more](#)

main.py > ask_claude

```
1 import boto3
2
3 def ask_claude(prompt, system="You are a helpful assistant."):
4     client = boto3.client('bedrock-runtime', region_name='us-east-1')
5     response = client.converse(
6         modelId="anthropic.claude-3-haiku-20240307-v1:0",
7         messages=[{"role": "user", "content": [{"text": prompt}]}],
8         system=[{"text": system}],
9         inferenceConfig={"temperature": 0, "maxTokens": 200}
10    )
11    return response['output'][0]['message'][0]['content'][0]['text']
12
13 # Example usage
14 print(ask_claude("Hello, how are you?"))
15
```

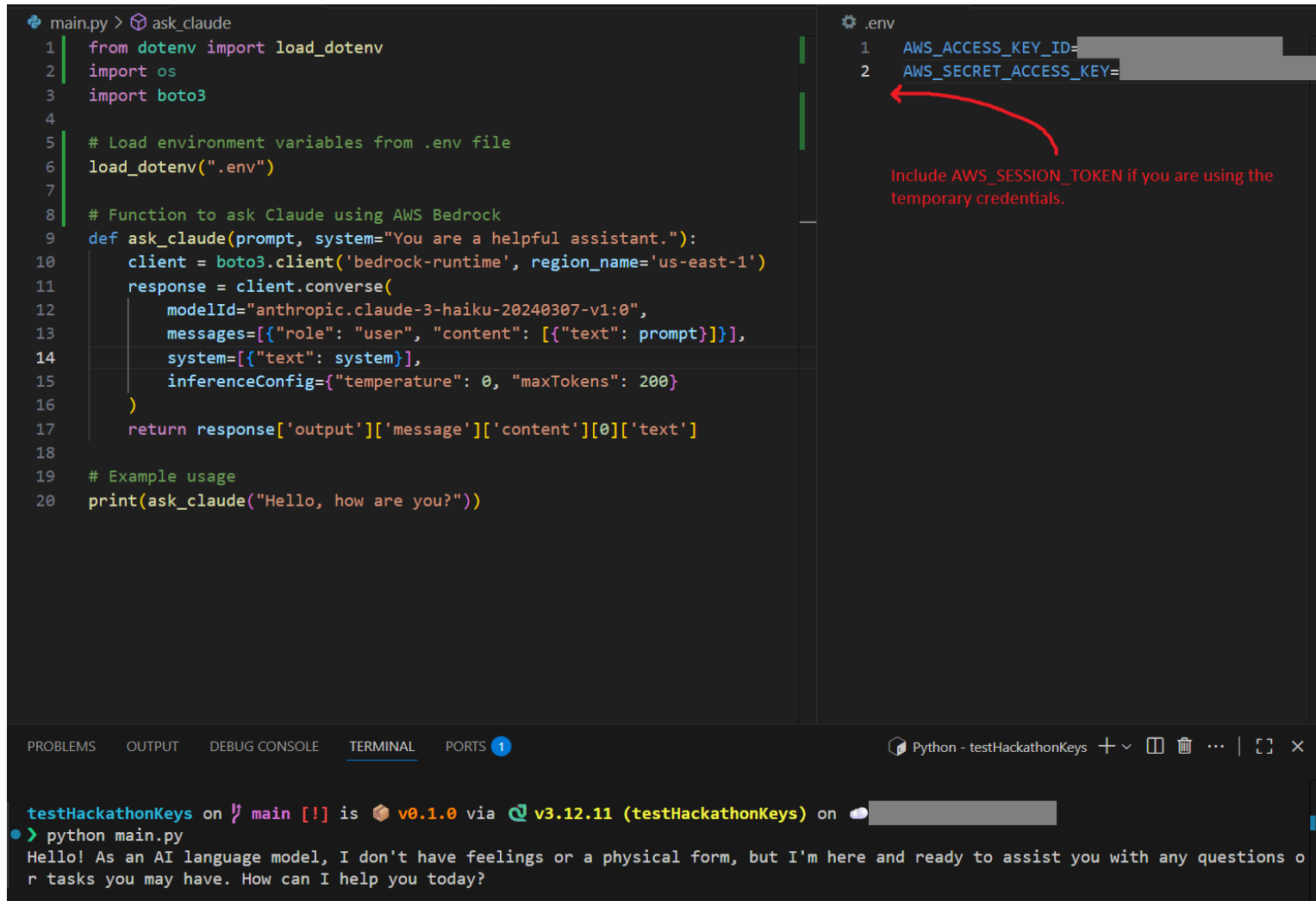
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1

```
testHackathonKeys on main [?] is v0.1.0 via v3.12.11 (testHackathonKeys) on [REDACTED]
• export AWS_ACCESS_KEY_ID="[REDACTED]"
  export AWS_SECRET_ACCESS_KEY="[REDACTED]"
  export AWS_SESSION_TOKEN="[REDACTED]"
```

```
testHackathonKeys on main [?] is v0.1.0 via v3.12.11 (testHackathonKeys) on [REDACTED]
```

```
• python main.py
Hello! As an AI language model, I don't have feelings or a physical form, but I'm here and ready to assist you with any questions or tasks you may have. How can I help you today?
```


#2: Use .env file



The screenshot shows a code editor with two files. The left file is `main.py` and the right file is `.env`.

main.py

```
1 from dotenv import load_dotenv
2 import os
3 import boto3
4
5 # Load environment variables from .env file
6 load_dotenv(".env")
7
8 # Function to ask Claude using AWS Bedrock
9 def ask_claude(prompt, system="You are a helpful assistant."):
10     client = boto3.client('bedrock-runtime', region_name='us-east-1')
11     response = client.converse(
12         modelId="anthropic.claude-3-haiku-20240307-v1:0",
13         messages=[{"role": "user", "content": [{"text": prompt}]],
14         system=[{"text": system}],
15         inferenceConfig={"temperature": 0, "maxTokens": 200}
16     )
17     return response['output'][0]['message'][0]['content'][0]['text']
18
19 # Example usage
20 print(ask_claude("Hello, how are you?"))
```

.env

```
1 AWS_ACCESS_KEY_ID=
2 AWS_SECRET_ACCESS_KEY=
```

A red arrow points from the text "Include AWS_SESSION_TOKEN if you are using the temporary credentials." to the `AWS_SECRET_ACCESS_KEY` line in the `.env` file.

The terminal at the bottom shows the command `python main.py` being executed, and the output is "Hello! As an AI language model, I don't have feelings or a physical form, but I'm here and ready to assist you with any questions or tasks you may have. How can I help you today?"

Recommended approach for managing secrets as environment variables in Python projects is to use libraries like python-dotenv.

Place your access keys in the file `./env`.

⚠ Do not commit your `.env` file to your git repo.

Tips

Monitor your ongoing costs

Monitor costs regularly

The screenshot displays the AWS Innovation Sandbox dashboard. At the top, it says "Welcome to Innovation Sandbox on AWS" with an information icon. Below this is a blue "Request lease" button. The main section is titled "My Leases (1)" with a refresh icon and a link to "View a list of your leases". A card for "Hackathon 2026" is shown, marked as "Active" and with a "Login to account" button. It lists the "AWS Account ID" as a redacted black box and the "Expiry" as "in 1 month". At the bottom, a "Budget" section features a light purple progress bar at 0%, with the text "\$0 of \$30" below it.

See the budget bar? Monitor it constantly! Note that there will be a slightly reporting lag (over a few hours), so do not expect charges to immediately be reflected on this indicator.

Note that once it exceeds \$20 (not \$30), access to your account will be automatically revoked.

Then why does the bar show \$30? If the bar is filled, then your AWS account and its resources will be terminated.

Stretching your \$20

Pick your services well

- **If you are spinning any of these up you are probably doing something wrong — they'll eat \$20 in a day, idle:**
 - **✗ OpenSearch (Serverless or domain)** — minimum always-on capacity costs *hundreds/month*. Use S3 Vectors.
 - **✗ SageMaker real-time endpoints** — bill per hour, non-stop, even with zero traffic. Use Bedrock instead.
 - **✗ NAT Gateway** — ~\$1/day just to exist, plus data charges. Don't put Lambda in a VPC unless you must; if you do, use VPC endpoints, not NAT.
 - **✗ Load balancers (ALB/NLB)** — ~\$0.50/day just for existing. You don't need one.
 - **✗ EC2 / RDS / always-on anything** — if you're launching a server, stop and rethink.
 - **✗ Bedrock Provisioned Throughput** — it's an hourly/monthly commitment worth thousands. On-demand only.

Go for Serverless

- **Build on this stack (all pay-per-use):**
- **Amazon Bedrock** for all AI — no servers, pay per token. Start with the cheap models (Amazon Nova Micro/Lite, Claude Haiku); only reach for bigger models when you actually need them.
- **AWS Lambda** for your backend (+ Lambda Function URLs, skip API Gateway if you can).
- **DynamoDB** on-demand for data — never "provisioned capacity."
- **S3** for files, and **S3 Vectors** for your knowledge base.
- **Bedrock Knowledge Bases** point straight at S3 Vectors — no cluster to run.
- See <https://serverlessland.com/> for some inspirations

Troubleshooting

Common problems

Not being in the correct region

The screenshot shows the AWS Management Console interface for Amazon Bedrock. At the top, the AWS logo and a search bar are visible. The top right corner displays the account ID and the current region, 'Asia Pacific (Sydney)'. A red arrow points to this region dropdown, with the text 'Wrong region!' written next to it. Below the navigation bar, the left sidebar shows the 'Amazon Bedrock' menu with options like 'Discover', 'Test', 'Infer', and 'Tune'. The main content area displays three red error bars, each stating 'Access denied to bedrock:ListFoundationModels' or 'Access denied to bedrock:ListInferenceProfiles'. Each error bar includes a 'Diagnose with Amazon Q' button. Below the error bars, the 'Overview' section is visible, featuring a 'Model catalog' card that describes the service and provides a link to 'View Model catalog'.

If you see multiple 'Access denied...' error bars, check if you are in the correct AWS Region at the top right. Make sure you are in us-east-1.